🚀 C++ Cheatsheet

🔰 1. Basics

- **Data Types**: `int`, `float`, `double`, `bool`, `char`, `void`
- **Modifiers**: `short`, `long`, `unsigned`, `signed`
- **Control Structures**: `if`, `else`, `switch`, `for`, `while`, `do-while`
- **Operators**:
  - Arithmetic: `+`, `-`, `*`, `/`, `%`
  - Logical: `&&`, `||`, `!`
  - Relational: `==`, `!=`, `<`, `>`, `<=`, `>=`
  - Bitwise: `&`, `|`, `^`, `~`, `<<`, `>>`
  - Assignment: `=`, `+=`, etc.
  - Scope Resolution: `::`

🧱 2. OOP Concepts

- **Class**: User-defined data type
- **Object**: Instance of a class
- **Access Specifiers**: `public`, `private`, `protected`
- **Encapsulation**, **Abstraction**, **Inheritance**, **Polymorphism**

📦 Class Example

```
class Person {
public:
    string name;
    void greet() { cout << "Hello, " << name; }
};
```

👨‍👩‍👧 Inheritance

```
class Student : public Person {
public:
    int id;
};
```

🔁 Polymorphism

- Compile-time: Function overloading, Operator overloading
- Runtime: Virtual functions

🧩 3. Advanced Features

📚 Templates

```
template<typename T>
T add(T a, T b) {
    return a + b;
}
```

🧠 Smart Pointers (`<memory>`)

- `unique_ptr`, `shared_ptr`, `weak_ptr`

🧵 Multithreading (`<thread>`)

```
#include <thread>
void func() { ... }
std::thread t(func);
t.join();
```

📜 Lambda Functions

```
auto add = [](int x, int y) { return x + y; };
```

🛠 4. STL (Standard Template Library)

🧺 Containers

- `vector`, `deque`, `list`
- `set`, `map`, `unordered_set`, `unordered_map`
- `stack`, `queue`, `priority_queue`

🧰 Algorithms (`<algorithm>`)

- `sort()`, `reverse()`, `find()`, `count()`, `accumulate()`
- Use with iterators

🧭 Iterators

```
for (auto it = v.begin(); it != v.end(); ++it) { ... }
```

🧃 5. Input/Output

```
#include <iostream>
using namespace std;

cin >> a;
cout << a;
getline(cin, str); // For strings with spaces
```

🎯 File I/O

```
#include <fstream>
ifstream in("input.txt");
ofstream out("output.txt");
```

⚙️ 6. Memory Management

- `new` and `delete`
- Avoid raw pointers; use smart pointers

🧪 7. Exception Handling

```
try {
    throw 10;
} catch (int e) {
    cout << "Caught " << e;
}
```

📦 8. Namespaces

```
namespace math {
    int add(int a, int b) { return a + b; }
}
using namespace math;
```

🧠 9. Miscellaneous

- `constexpr` : Compile-time evaluation
- `enum class` : Scoped enum
- `auto` : Type inference
- `decltype` : Type deduction
- `static_assert` : Compile-time assertion

🔍 10. Compilation

- Compile: `g++ file.cpp -o output`

- With flags: `-Wall`, `-O2`, `-std=c++17`, `-g`